System Testing and Evaluation Report

AU Roadmap Project

Project Name: Adelaide University Program Roadmap

Course: Industry Research Project

Semester: Semester 1, 2026

Group Code: C261W-4037

Team Members:

Jialao Liu

Yuanxin Huang

Yilin Peng

Vatsal Thanki

Reported by: Jialao Liu

Academic Mentor: Wenhao Liang

Version: 1.0

Document Status: Final

Last Updated: 09 May 2026

Test Environment	Local Development — http://localhost:8080/api

Tools Used	Postman, Python (smoke_test.py), JMeter (3 scenarios executed)

Overall Result	PASS — 19/19 test cases passed (10 Postman + 9 automated) + 3 JMeter scenarios, 1,200 requests, 0% error

1. Executive Summary

This report documents the system testing and evaluation work completed for the AU Roadmap project, a university industry-research prototype providing students with program roadmaps, community connections, alumni networking, and administrative management tools. This report primarily covers backend API functional validation, automated smoke testing, and local JMeter-based performance testing. Testing was carried out in the local development environment on 09 May 2026.

Three levels of testing were conducted:

- Manual functional API testing using Postman (10 test cases)
- Automated API smoke testing using a Python script (9 test cases)
- Performance testing using JMeter across three public API scenarios (1,200 total requests)

All 19 Postman and Python test cases passed. Core public, student-authenticated, and admin-authenticated endpoints responded correctly. Validation and access-control negative tests confirmed the system rejects invalid input and unauthorised requests with appropriate HTTP status codes. All three JMeter scenarios completed with 0% error rate and sub-10 ms average response times in the local development environment.

Testing Method	Test Cases / Requests	Result
Postman — Manual API Testing	10 test cases	**10 / 10 Passed**
Python — Automated Smoke Tests	9 test cases	**9 / 9 Passed**
JMeter — Smoke Baseline Load	200 requests (10T × 5L × 4EP)	**0 errors — Avg 4.00 ms, Max 146 ms**
JMeter — Moderate Public API Load	640 requests (20T × 8L × 4EP)	**0 errors — Avg 2.39 ms, Max 52 ms**
JMeter — Focused Roadmap & Search	360 requests (15T × 12L × 2EP)	**0 errors — Avg 2.53 ms, Max 25 ms**

2. Test Scope

Note: This report primarily covers backend API functional validation, automated smoke testing, and local JMeter-based performance testing. Frontend UI, cross-browser compatibility, and formal accessibility testing were not included in this testing round.

2.1 In Scope

- Backend public REST APIs (health, programs, roadmap, search)
- Backend student-protected REST APIs (auth/me, community connections)
- Backend admin-protected REST APIs (admin stats, course list)
- Authentication and authorisation behaviour (JWT-based)
- Unauthorised access rejection (missing/invalid token)
- Server-side input validation (discussion creation with empty fields)
- Automated regression smoke testing via Python script
- Performance test plan preparation via JMeter

2.2 Out of Scope

- Browser-based end-to-end UI automation
- High-scale performance benchmarking in staging or production environments
- Formal accessibility audit using dedicated tools (e.g. axe, Lighthouse)
- Full usability testing with external participants
- Production deployment or staging environment testing
- CAPTCHA-based (Cloudflare Turnstile) login and registration flow automation

Note: Because the registration and login flows require Cloudflare Turnstile CAPTCHA, protected-route testing was conducted using valid local development JWT tokens for a student user and an admin user rather than live authentication flows.

3. Test Environment

Parameter	Value
Project	Adelaide University Program Roadmap
Test Date	09 May 2026
Environment Type	Local Development
Backend Base URL	http://localhost:8080/api
Backend Runtime	Node.js (Express)
Database	Local MySQL instance — configured via backend/.env
Authentication Method	JWT (HS256) — tokens generated from local JWT_SECRET for protected route testing
Operating System	macOS (Darwin)

3.1 Entry Criteria

- Backend server started successfully (npm run dev)
- Database reachable with seed data present
- backend/.env configured with valid DB credentials and JWT_SECRET
- Valid local student and admin JWT tokens available for protected route testing

3.2 Exit Criteria

- All selected Postman requests return the expected HTTP status codes
- All automated smoke tests pass (9/9)
- JMeter test plan is created and structurally validated
- Testing evidence is documented in this formal report

4. Testing Tools

4.1 Postman (Manual Functional API Testing)

Postman Desktop was used for manual functional API testing. A dedicated collection and environment file were created to exercise public, student-protected, admin-protected, and negative test cases.

- Collection file: tests/api/au-roadmap.postman_collection.json
- Environment file: tests/api/au-roadmap.local.postman_environment.json
- Environment variables used: baseUrl, studentToken, adminToken

4.2 Python — Automated Smoke Test Script

A lightweight Python script (smoke_test.py) was written to automate a compact but representative regression suite. The script generates local JWT tokens from the backend .env JWT_SECRET, making it self-contained without requiring manual token configuration. It can be re-executed after any backend change to confirm no regression has occurred.

- Script: tests/api/smoke_test.py
- Runtime: Python 3 (stdlib only — no third-party dependencies)
- Token strategy: auto-generated HS256 JWTs using JWT_SECRET from backend/.env
- Execution: python3 tests/api/smoke_test.py

4.3 JMeter — Performance Testing

Apache JMeter was used to execute three performance test scenarios against the local backend. All three scenarios completed successfully with 0% error rate across 1,200 total requests. HTML reports were generated for each scenario.

- Scenario 1 — Smoke Baseline: tests/performance/au-roadmap-public-api-smoke.jmx
- Scenario 2 — Moderate Load: tests/performance/au-roadmap-public-api-moderate-load.jmx
- Scenario 3 — Focused Roadmap & Search: tests/performance/au-roadmap-search-roadmap-focus.jmx
- Results: tests/performance/results/{smoke,moderate,focus}/results.jtl
- HTML Reports: tests/performance/results/{smoke,moderate,focus}/report/index.html

5. Test Strategy

The test strategy combined three complementary testing levels:

5.1 Manual Functional Testing (Postman)

Manual API tests verified that each endpoint returns the correct HTTP status code and a well-formed JSON response. Tests covered positive cases (valid requests), authentication cases (valid JWT tokens), and negative cases (missing tokens, invalid input). Postman test scripts embedded in each request provided automated assertion results for each execution.

5.2 Automated Smoke Testing (Python)

The smoke test script provided a fast, repeatable regression check. Unlike the Postman collection, the Python script requires no GUI and can be run from the command line after any code change. It covers the same critical API surface as the Postman tests and confirms that no regression has been introduced.

5.3 Performance Testing (JMeter)

Three JMeter test plans were executed against public endpoints to evaluate lightweight and moderate local performance behaviour. By targeting read-only public endpoints, the plans were safe to run repeatedly without database write side-effects. Results provide a credible performance baseline for the prototype within the local development environment.

6. Postman Functional Test Execution

All 10 Postman test cases were executed manually against the local backend on 09 May 2026. The results are summarised below.

ID	Test Case	Method	Endpoint	Type	Expected	Result
TC-01	Health Check	GET	/health	Public	200 OK — { "status": "ok" }	**Passed**
TC-02	List Programs	GET	/programs?page=1&limit=5	Public	200 OK — success: true, data array present	**Passed**
TC-03	Program Roadmap	GET	/programs/3/roadmap	Public	200 OK — data.roadmap exists	**Passed**
TC-04	Search Programs	GET	/search?q=computer&type=programs&limit=5	Public	200 OK — success: true, programs array present	**Passed**
TC-05	Get Current User	GET	/auth/me	Student Protected	200 OK — authenticated user profile	**Passed**
TC-06	List Connections	GET	/community/connections	Student Protected	200 OK — connections array	**Passed**
TC-07	Discussion Validation Check	POST	/discussions/create	Student Protected	400 Bad Request — VALIDATION_ERROR	**Passed**
TC-08	Admin Stats	GET	/admin/stats	Admin Protected	200 OK — admin stats object	**Passed**
TC-09	List Courses	GET	/admin/courses	Admin Protected	200 OK — courses array	**Passed**
TC-10	Unauthorised Access Check	GET	/auth/me (no token)	Negative	401 Unauthorized — UNAUTHORIZED error	**Passed**

6.1 Key Observations

- TC-01 to TC-04: All four public endpoints responded correctly with HTTP 200 and well-formed JSON.
- TC-05 to TC-07: Student-protected endpoints accepted valid JWT tokens and returned expected data. TC-07 confirmed that the Discussion validation fix (DEF-035) is functioning — empty submissions are rejected with a controlled 400 error.
- TC-08 to TC-09: Admin-protected endpoints correctly granted access to the admin user token and returned dashboard statistics and course data.
- TC-10: The unauthorised access check confirmed that the authentication middleware blocks requests without a token and returns a structured 401 response.

6.2 Token Strategy

Because login and registration currently require Cloudflare Turnstile CAPTCHA, live authentication automation was not used. Instead, valid HS256 JWT tokens were generated locally using the project's JWT_SECRET and configured as Postman environment variables (studentToken, adminToken). This approach allowed full coverage of protected endpoints without circumventing security — the tokens are signed with the same secret used by the backend.

7. Python Automated Smoke Test Results

The Python smoke test script was executed against the local backend using the following command:

```
python3 tests/api/smoke_test.py
```

The script auto-generated JWTs from backend/.env JWT_SECRET — no manual token configuration was required. All 9 tests passed.

#	Test	Result
1	GET /health	**PASS**
2	GET /programs	**PASS**
3	GET /programs/3/roadmap	**PASS**
4	GET /search	**PASS**
5	GET /auth/me (unauthorized)	**PASS**
6	GET /auth/me as student	**PASS**
7	GET /community/connections as student	**PASS**
8	POST /discussions/create validation	**PASS**
9	GET /admin/stats as admin	**PASS**

Summary: 9/9 tests passed.

7.1 Script Design Notes

- Written in Python 3 using stdlib only — no pip dependencies required.
- Reads JWT_SECRET from backend/.env and generates HS256 tokens at runtime.
- Supports environment variable overrides (AU_TEST_BASE_URL, AU_TEST_STUDENT_TOKEN, AU_TEST_ADMIN_TOKEN) for CI pipeline integration.
- Exits with code 0 on full pass, code 1 on any failure — compatible with CI/CD pass/fail gates.

8. JMeter Performance Test Results

Three JMeter performance test scenarios were executed against the local backend on 09 May 2026. All scenarios completed successfully with 0% error rate across a combined total of 1,200 requests. HTML reports were generated for each scenario.

8.1 Combined Results Overview

Scenario	Requests	Threads	Avg (ms)	Max (ms)	Error Rate	Result
Smoke Baseline Load	200	10	4.00	146	0.00%	PASS
Moderate Public API Load	640	20	2.39	52	0.00%	PASS
Focused Roadmap & Search	360	15	2.53	25	0.00%	PASS
Total	**1,200**	—	—	—	**0.00%**	**ALL PASS**

8.2 Scenario 1 — Smoke Baseline Load

Parameter	Value
Plan File	tests/performance/au-roadmap-public-api-smoke.jmx
Purpose	Verify stable baseline performance under light concurrent public traffic
Endpoints	GET /api/health | GET /api/programs | GET /api/programs/3/roadmap | GET /api/search
Config	10 threads, ramp-up 10 s, 5 loops
Total Requests	200
Avg / Max	4.00 ms average | 146 ms maximum
Error Rate	0.00%
Result Files	tests/performance/results/smoke/results.jtl | .../smoke/report/index.html

Per-endpoint breakdown — Smoke Baseline

Endpoint	Count	Avg (ms)	Min (ms)	Max (ms)	Errors	Status
GET /api/health	50	3.12	1	81	0	PASS
GET /api/programs	50	6.24	2	146	0	PASS
GET /api/programs/3/roadmap	50	4.08	1	22	0	PASS
GET /api/search	50	2.58	1	6	0	PASS

8.3 Scenario 2 — Moderate Public API Load

Parameter	Value
Plan File	tests/performance/au-roadmap-public-api-moderate-load.jmx
Purpose	Simulate a heavier public demo workload with more users and more total requests
Endpoints	GET /api/health | GET /api/programs | GET /api/programs/3/roadmap | GET /api/search
Config	20 threads, ramp-up 15 s, 8 loops

Total Requests	640
Avg / Max	2.39 ms average | 52 ms maximum
Error Rate	0.00%
Result Files	tests/performance/results/moderate/results.jtl | .../moderate/report/index.html

Per-endpoint breakdown — Moderate Load

Endpoint	Count	Avg (ms)	Min (ms)	Max (ms)	Errors	Status
GET /api/health	160	1.28	0	20	0	**PASS**
GET /api/programs	160	2.71	1	7	0	**PASS**
GET /api/programs/3/roadmap	160	3.36	1	52	0	**PASS**
GET /api/search	160	2.23	0	4	0	**PASS**

8.4 Scenario 3 — Focused Roadmap and Search Load

Parameter	Value
Plan File	tests/performance/au-roadmap-search-roadmap-focus.jmx
Purpose	Focus testing on the two most data-rich public endpoints: roadmap retrieval and keyword search
Endpoints	GET /api/programs/3/roadmap | GET /api/search
Config	15 threads, ramp-up 10 s, 12 loops
Total Requests	360
Avg / Max	2.53 ms average | 25 ms maximum
Error Rate	0.00%
Result Files	tests/performance/results/focus/results.jtl | .../focus/report/index.html

Per-endpoint breakdown — Focused Load

Endpoint	Count	Avg (ms)	Min (ms)	Max (ms)	Errors	Status
GET /api/programs/3/roadmap	180	3.04	1	25	0	**PASS**
GET /api/search	180	2.02	1	4	0	**PASS**

8.5 Performance Observations

Important: All performance results in this section apply to the local development environment only and should not be interpreted as production-scale benchmarks. Network latency, concurrent user volume, and database load in a production deployment would differ significantly.

- All three scenarios achieved 0.00% error rate — the backend handled all 1,200 requests without a single failure.

- Average response times across all scenarios were well under 10 ms, indicating the local Node.js + MySQL stack handles these public endpoints efficiently.
- The highest single-request latency observed was 146 ms (GET /api/programs, smoke scenario), likely a cold-start or JVM warm-up spike typical in the first few requests of a new test run. All subsequent requests in that scenario were well below this value.
- In the moderate load scenario (20 threads), average response times actually decreased compared to the smoke baseline (2.39 ms vs 4.00 ms), suggesting the server benefits slightly from a warmed-up connection pool under light concurrency.
- The roadmap endpoint (GET /api/programs/3/roadmap) consistently had the highest average latency across all scenarios (3.04–4.08 ms), which is expected given it performs a multi-table JOIN for roadmap data.
- The search endpoint (GET /api/search) remained the fastest across all scenarios (2.02–2.58 ms average).

9. Defect Summary

Defect identification and tracking for the AU Roadmap project was maintained separately in the project Defect Report. A total of 45 defects were recorded across all tracked development phases, and all 45 have been resolved at the time of this report.

Sprint / Phase	Total	Fixed	Open	Recurrences
Initial (Pre-Sprint)	6	6	0	0
Sprint 0318 → 0414	16	16	0	4
Sprint 0428	5	5	0	0
Sprint 0507	6	6	0	0
Additional Fixes	12	12	0	0
Total	**45**	**45**	**0**	**4**

Defects verified by testing in this round: TC-07 (Discussion Validation Check) directly confirmed that DEF-035 (Discussion validation not taking effect) is resolved — the server now returns 400 VALIDATION_ERROR for empty submissions. TC-10 (Unauthorised Access Check) confirmed that authentication middleware (relevant to DEF-001, DEF-013) functions correctly.

10. Risks and Limitations

Limitation	Impact	Mitigation Applied
Cloudflare Turnstile CAPTCHA on login/register prevents automated auth-flow testing	Cannot exercise registration and login endpoints in automated tests	Valid local JWT tokens used for all protected-route testing
JMeter tested against local environment only — not production or staging scale	Results reflect single-machine performance and do not represent production load capacity	Results clearly scoped to local dev; three scenarios provide comparative baseline data
No browser-based UI automation suite implemented	Frontend rendering and user flow correctness not programmatically verified	Manual browser testing and review completed as part of development
No formal accessibility tool audit (e.g. axe, Lighthouse)	WCAG compliance not formally verified beyond code-level contrast fix (DEF-002)	Manual accessibility review conducted during development

11. Frontend Validation Note

Frontend pages were manually reviewed in the browser throughout the development and refinement process. Visual correctness, navigation behaviour, and key user flows — including the student portal, roadmap display, community hub, and admin dashboard — were verified through direct browser interaction.

However, no dedicated automated UI testing framework (such as Playwright or Cypress) was included in this testing round. As a result, frontend validation is not formally covered by automated test cases in this report. Manual browser review was the primary method used to confirm frontend behaviour.

Recommended next step: *implement a Playwright or Cypress end-to-end test suite to automate critical user flows and provide repeatable frontend regression coverage.*

12. Conclusion

The AU Roadmap prototype has been tested across three levels in the local development environment: manual functional API testing, automated smoke testing, and JMeter-based performance testing. All 19 Postman and Python test cases passed. The system demonstrates correct behaviour for public access, authenticated student access, authenticated admin access, unauthorised access rejection, and server-side input validation. All three JMeter performance scenarios completed with 0% error rate.

The following testing deliverables have been produced:

- Postman collection and environment for repeatable manual API testing
- Python smoke test script for automated regression checking with zero external dependencies
- Three JMeter performance test scenarios with full execution results and HTML reports
- This formal test report documenting all results, environment, and methodology

For a university industry-research prototype, this represents a strong and credible testing foundation. Recommended improvements for the next phase:

- Extend the Python smoke test suite to cover additional edge cases and negative scenarios
- Implement UI-level browser automation (e.g. Playwright or Cypress)

- Conduct a formal accessibility audit using Lighthouse or axe-core
- Re-run JMeter against a staging or production environment to gather production-scale performance data

Appendix: Test Assets Reference

A. Test File Inventory

File	Purpose
tests/api/au-roadmap.postman_collection.json	Postman API test collection — public, student, admin, negative test cases
tests/api/au-roadmap.local.postman_environment.json	Postman local environment variables (baseUrl, studentToken, adminToken)
tests/api/smoke_test.py	Python automated smoke test script — 9 regression checks, self-contained JWT generation
tests/performance/au-roadmap-public-api-smoke.jmx	JMeter Scenario 1 — Smoke baseline load (10T × 5L × 4EP = 200 requests)
tests/performance/au-roadmap-public-api-moderate-load.jmx	JMeter Scenario 2 — Moderate public API load (20T × 8L × 4EP = 640 requests)
tests/performance/au-roadmap-search-roadmap-focus.jmx	JMeter Scenario 3 — Focused roadmap & search load (15T × 12L × 2EP = 360 requests)
tests/performance/results/smoke/results.jtl	Raw JMeter result data — Smoke Baseline
tests/performance/results/moderate/results.jtl	Raw JMeter result data — Moderate Load
tests/performance/results/focus/results.jtl	Raw JMeter result data — Focused Load
tests/performance/results/smoke/report/index.html	JMeter HTML report — Smoke Baseline
tests/performance/results/moderate/report/index.html	JMeter HTML report — Moderate Load
tests/performance/results/focus/report/index.html	JMeter HTML report — Focused Load

b. References

- IEEE 829 — Standard for Software and System Test Documentation
- ISTQB Foundation Level Syllabus — Test Planning and Reporting Guidance
- Apache JMeter Documentation — https://jmeter.apache.org
- Postman Documentation — https://learning.postman.com